



TAWHIDIC EPISTEMOLOGY **UMMATIC EXCELLENCE**
LEADING THE WAY **LEADING THE WORLD**
KHALĪFAH • AMĀNAH • IQRA' • RAHMATAN UL-ĀLAMĪN

INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA

KULLIYAH OF ENGINEERING

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2 2025/2026

WEEK 8: HuskyLensAI Vision and Real-Time Color Detection

SECTION 1

GROUP 11

**INSTRUCTOR: DR. WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR.
ZULKIFLI BIN ZAINAL ABIDIN**

Date of Experiment: Wednesday, 29th April 2026

Date of Submission: Wednesday, 6th May 2026

NO.	NAME	MATRIC NO.
1.	NURUL ANIS BINTI MOHD KHAIRI	2419788
2.	WAN NUR ATHIRAH BINTI MEOR MOHD REDZUAN	2320102
3.	SITI NURKHALISAH BINTI NAHZAMRI	2328482

ABSTRACT

This experiment aims to investigate the use of the HuskyLens AI Vision sensor integrated with an Arduino microcontroller for real-time object and color detection. The methodology consists of two tasks: object recognition with LED control and multi-color recognition with RGB LED output. In Task 1, the system uses if-else logic to detect a trained object (“Object 1”) and control an LED based on its presence. In Task 2, the HuskyLens identifies multiple color IDs, which are mapped to corresponding RGB LED outputs and displayed on the Serial Monitor.

The key findings show that the HuskyLens successfully performs onboard image classification and communicates detection results effectively to the Arduino for real-time response. However, detection accuracy is affected by environmental conditions such as ambient lighting and shadows, which may reduce confidence levels. In conclusion, the experiment demonstrates the effectiveness of AI vision integration in embedded systems, where visual inputs are translated into direct physical outputs, enabling interactive and responsive applications.

TABLE OF CONTENTS

INTRODUCTION	5
TASK 1: HUSKEYLENS SETUP AND BASIC AI PRACTICE	5
PART A: OBJECT RECOGNITION CONTROL	5
1.1.0 MATERIALS AND EQUIPMENTS	5
1.1.1 EXPERIMENTAL SETUP	6
1.1.2 METHODOLOGY	6
a. Circuit Assembly	6
b. Programming and Uploading	6
c. Programming Logic.....	7
d. Coding	7
1.1.3 DATA ANALYSIS	8
1.1.4 RESULT	8
1.1.5 DISCUSSION	8
PART B: MULTI COLOR RECOGNITION.....	9
1.2.0 MATERIALS AND EQUIPMENTS	9
1.2.1 EXPERIMENTAL SETUP	9
1.2.2 METHODOLOGY	9
a. Circuit Assembly	9
b. Programming and uploading	10
c. Programming Logic.....	10
d. Coding	10
1.2.3 DATA ANALYSIS	11
1.2.4 RESULT	12
1.2.5 DISCUSSION	12
TASK 2: INTEGRATED RGB COLOR DISPLAY	13
2.0 MATERIALS AND EQUIPMENTS	13
2.1 EXPERIMENTAL SETUP	13
2.2 METHODOLOGY	13
a. Circuit Assembly	14
b. Programming and uploading	14
c. Programming Logic.....	14
d. Coding	14
2.3 DATA ANALYSIS	16
2.4 RESULT	16
2.5 DISCUSSION	17
3.0 CONCLUSION.....	18

4.0 RECOMMEENDATION	18
5.0 REFERENCES	19
6.0 ACKNOWLEDGEMENT	19
7.0 STUDENTS DECLARATION	20

INTRODUCTION

Artificial Intelligence (AI) vision systems have become increasingly important in modern engineering applications, particularly in automation, robotics, and smart sensing systems. By enabling machines to interpret and respond to visual data, AI cameras such as the HuskyLens simplify complex image processing tasks. When integrated with microcontrollers like Arduino, these systems allow real-time interaction between visual inputs and physical outputs, making them highly suitable for embedded system applications.

In Task 1, the focus is on basic object and colour recognition using the HuskyLens. The system is programmed to detect a trained object (“Object 1”) and control an LED based on its presence using simple if-else logic. In addition, multi-colour recognition is implemented where different colour IDs are identified and displayed through the Serial Monitor. This task demonstrates how the HuskyLens processes visual data and communicates recognition results to the Arduino for decision-making.

In Task 2, the system is further enhanced by integrating an RGB LED to provide visual output based on detected colour IDs. Instead of displaying results on the Serial Monitor, the detected colours are translated into corresponding RGB light outputs, creating a more interactive and responsive system. This task highlights the practical application of AI vision in real-time control systems, where detected inputs are directly converted into meaningful physical feedback.

TASK 1: HUSKEYLENS SETUP AND BASIC AI PRACTICE

PART A: OBJECT RECOGNITION CONTROL

1.1.0 MATERIALS AND EQUIPMENTS

- Arduino Uno
- USB Cable
- Gravity HuskyLens AI Vision Camera
- Standard LED x1

- 220 Ω Resistors x1
- Jumper Wires
- Breadboard

1.1.1 EXPERIMENTAL SETUP

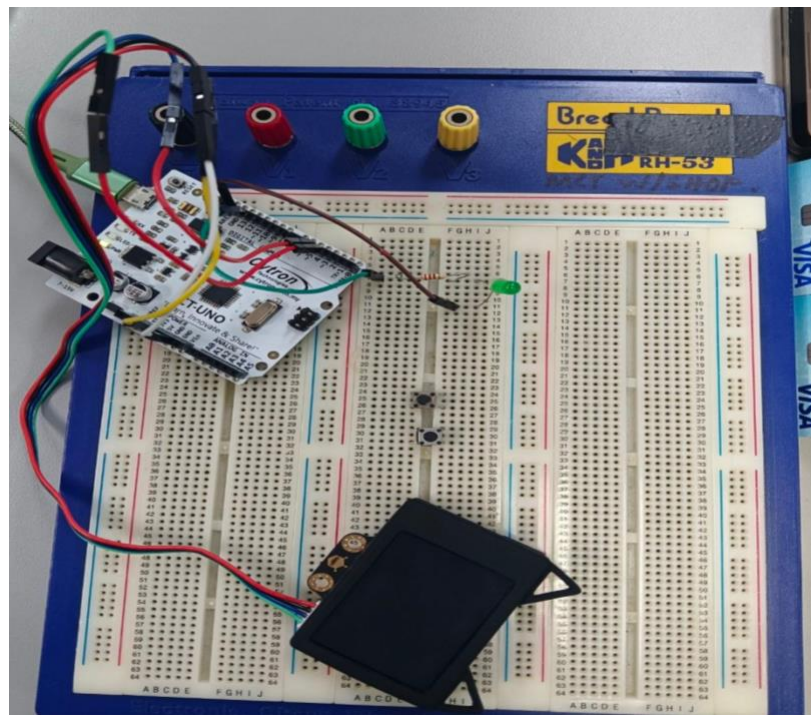


Figure 1.1 shows the assembly circuit for task 1.1

1.1.2 METHODOLOGY

a. Circuit Assembly

- VCC is being connected to 5V at Arduino
- GND is connected to GND at Arduino
- TX is connected to D4, and RX is connected to D5
- The cathode of LED light is connected to GND, and anode of LED light is connected to analog pin 13
- The anode of resistor is connected to pin 13 and the cathode is connected to the GND

b. Programming and Uploading

- For the husky lens configurations, the dial button is being dial until the general setting is being found and select it.
- The protocol type is being dialled, and it is being pressed down.
- The serial 9600 is being pressed down and dial it to “Save and return”
- The Arduino IDE is being downloaded and connect it to COM 9.

c. Programming Logic

- The if-else logic is being used, if the object detected can categorized as “Object 1” the LED will turn on.
- And if the object can’t be detected as “Object 1”, the LED will turn off.

d. Coding

```

Week_8_task_1.ino
1  #include "HUSKYLENS.h"
2  #include "SoftwareSerial.h"
3
4  HUSKYLENS huskylens;
5  SoftwareSerial mySerial(4, 5); // RX, TX
6  // Define the LED pin
7  const int ledPin = 13;
8
9  void setup() {
10     Serial.begin(115200);
11     mySerial.begin(9600);
12     // Initialize the LED pin as an output
13     pinMode(ledPin, OUTPUT);
14     digitalWrite(ledPin, LOW); // Start with LED off
15     // Wait for HuskyLens to connect
16     while (!huskylens.begin(mySerial)) {
17         Serial.println(F("Begin failed! Check wiring and Protocol (Serial 9600)"));
18         delay(500);
19     }
20     Serial.println("HuskyLens Ready!");
21 }
22
23 void loop() {
24     // 1. Request data from HuskyLens
25     if (!huskylens.request()) {
26         // If the request fails or sees nothing, turn the LED off
27         digitalWrite(ledPin, LOW);
28         return;
29     }
30     // 2. Check if any trained objects are in view
31     if (huskylens.available()) {
32         HUSKYLENSResult result = huskylens.read();
33         // 3. Check if the detected object is "Object 1"
34         if (result.command == COMMAND_RETURN_BLOCK && result.ID == 1) {
35             Serial.println("Object 1 detected! LED ON");
36             digitalWrite(ledPin, HIGH);
37         }
38         else {
39             // If a different object is seen, turn LED off
40             digitalWrite(ledPin, LOW);
41         }
42     }
43     else {
44         // 4. If no objects are available in the frame, turn LED off
45         digitalWrite(ledPin, LOW);
46     }
47 }

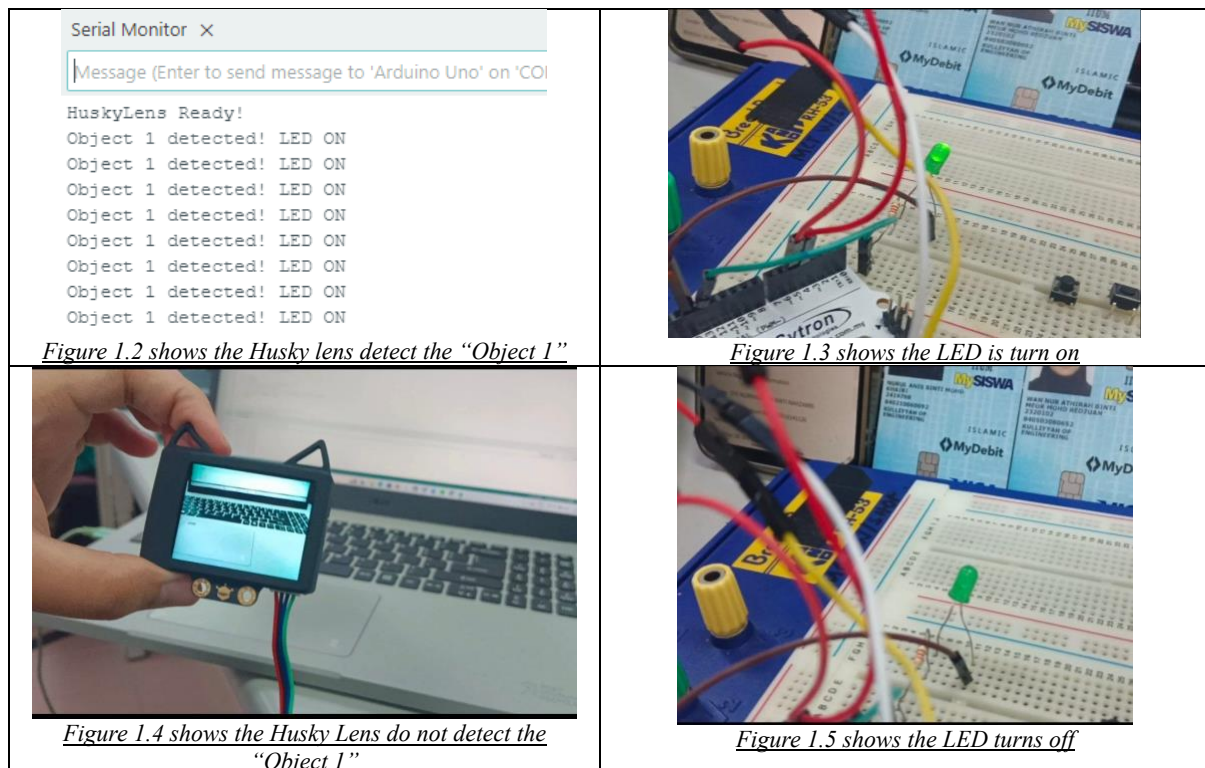
```

1.1.3 DATA ANALYSIS

Gravity HuskyLens AI Vision Camera	The LED
The Husky lens can detect the “Object 1”	Turns on
The Husky lens can’t detect the “Object 1”	Turns off

From the table above, we can see that when the Husky lens can detect the “object 1”, the LED will turn on and when it fails to detect the “object 1”, The LED will turn off. The system uses Object Recognition as the control trigger. Other than that, this task is also an open loop system because the LED is turn on when it detects the “object 1” and it does not have any feedback to the Husky lens to change its behaviour when it detects something.

1.1.4 RESULT



1.1.5 DISCUSSION

The results show that the system functions as intended, where the LED turns ON when “Object 1” is detected and turns OFF when no object is recognized. This confirms that

the implemented if-else logic operates correctly, allowing the Arduino to respond to signals from the HuskyLens in real time. The system successfully demonstrates basic object recognition as a control trigger for an external output device.

However, the system performance may be affected by external factors such as lighting conditions, distance, and object positioning, which can lead to inconsistent detection. Additionally, the system operates as an open-loop system, as there is no feedback mechanism to verify or correct the detection process. As a result, any detection errors cannot be automatically adjusted, which limits the overall reliability of the system.

PART B: MULTI COLOR RECOGNITION

1.2.0 MATERIALS AND EQUIPMENTS

- Arduino Uno
- USB Cable
- Gravity HuskyLens AI Vision Camera
- Jumper Wires

1.2.1 EXPERIMENTAL SETUP

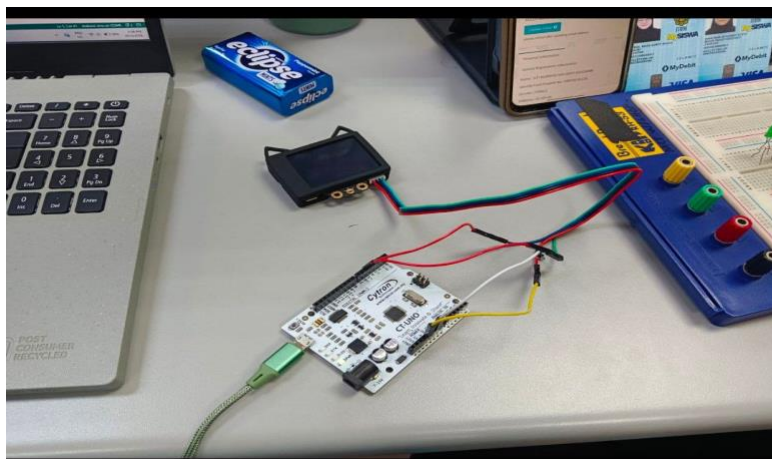


Figure 2.1 shows the experimental setup for task 1.2

1.2.2 METHODOLOGY

a. Circuit Assembly

- The VCC is connected to 5V at Arduino
- GND to Arduino GND at Arduino
- TX is connected to Arduino D4
- RX is connected to Arduino D5

b. Programming and uploading

- For the husky lens configurations, the dial button is being dial until the general setting is being found and select it.
- The protocol type is being dialled, and it is being pressed down.
- The serial 9600 is being pressed down and dial it to “Save and return”
- The Arduino IDE is being downloaded and connect it to COM 9

c. Programming Logic

- If the camera detects an object, it reads the specific result.ID; if the detected object is categorized as "Object 1" (Case 1), the LED turns Red.
- If the detected object is categorized as "Object 2" (Case 2), the LED turns Blue.
- If the detected object is categorized as "Object 3" (Case 3), the LED turns Green.

d. Coding

Week_8_task_1_part_b.ino

```
1  #include "HUSKYLENS.h"
2  #include "SoftwareSerial.h"
3
4  HUSKYLENS huskylens;
5  SoftwareSerial mySerial(4, 5); // RX, TX
6
7  void setup() {
8      Serial.begin(115200);
9      mySerial.begin(9600);
10
11     while (!huskylens.begin(mySerial)) {
12         Serial.println(F("Begin failed!"));
13         delay(500);
14     }
15     Serial.println("HuskyLens Color Recognition Ready!");
16 }
17
18 void loop() {
19     if (!huskylens.request()) return;
20
21     if (huskylens.available()) {
22         HUSKYLENSResult result = huskylens.read();
23
24         if (result.command == COMMAND_RETURN_BLOCK) {
25             // Evaluate the ID and print the corresponding color
26             switch (result.ID) {
27                 case 1:
28                     Serial.println("Color RED is detected");
29                     break;
30                 case 2:
31                     Serial.println("Color BLUE is detected");
32                     break;
33                 case 3:
34                     Serial.println("Color GREEN is detected");
35                     break;
36                 default:
37                     Serial.println("Unknown Color detected");
38                     break;
39             }
40         }
41     }
42 }
```

1.2.3 DATA ANALYSIS

Gravity HuskyLens AI Vision Camera	Output Serial monitor
The Husky lens can detect the “Color ID 2”	The serial shows “BLUE”
The Husky lens can detect the “Color ID 3”	The serial shows “GREEN”
The Husky lens can detect the “Color ID 1”	The serial shows “RED”

The output is being determined by triggering a specific output only when a confidence threshold is met for a specific ID. Based on the table, when the husky lens detect an object and recognised as “Color ID 2” the serial monitor will show “BLUE”. Next, when the husky lens detects an object known as “Color ID 3”, the serial monitor will show “GREEN”. Lastly when the husky lens detects the “Color ID 1”, the serial will show “RED”.

1.2.4 RESULT



Figure 2.2 shows the husky lens detect an object as “Color ID2”

```
Output  Serial Monitor  X
Message (Enter to send message to 'Arduino Uno' on 'COM
Color BLUE is detected
Color BLUE is detected
Color BLUE is detected
Color BLUE is detected
Color BLUE is detected
Color BLUE is detected
Color BLUE is detected
Color BLUE is detected
```

Figure 2.3 shows serial monitor detect the “Color ID3” as “BLUE”



Figure 2.4 shows the husky lens detect an object as “Color ID3”

```
Output  Serial Monitor  X
Message (Enter to send message to 'Arduino Uno' on 'COM
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
```

Figure 2.5 shows serial monitor detect the “Color ID3” as “GREEN”

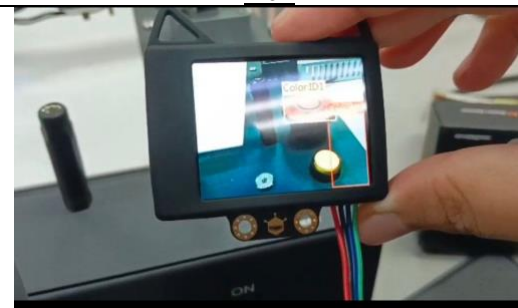


Figure 2.6 shows the husky lens detect an object as “Color ID1”

```
Output  Serial Monitor  X
Message (Enter to send message to 'Arduino Uno' on 'COM
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
Color GREEN is detected
```

Figure 2.6 shows serial monitor detect the “Color ID1” as “RED”

1.2.5 DISCUSSION

The implementation of multi-color recognition using the HuskyLens AI vision camera demonstrates an efficient way to offload complex visual processing from a standard microcontroller. By using the "Color Recognition" algorithm onboard the HuskyLens, the camera identifies specific color signatures and assigns them unique ID numbers. As seen in the code, the Arduino does not need to process raw pixel data. Instead, it simply listens for these IDs over a serial connection. The use of a switch-case structure is ideal for this application, as it allows for a clean, modular way to add more colors by simply defining new cases for additional IDs. This specific logic creates a responsive system where the Serial Monitor

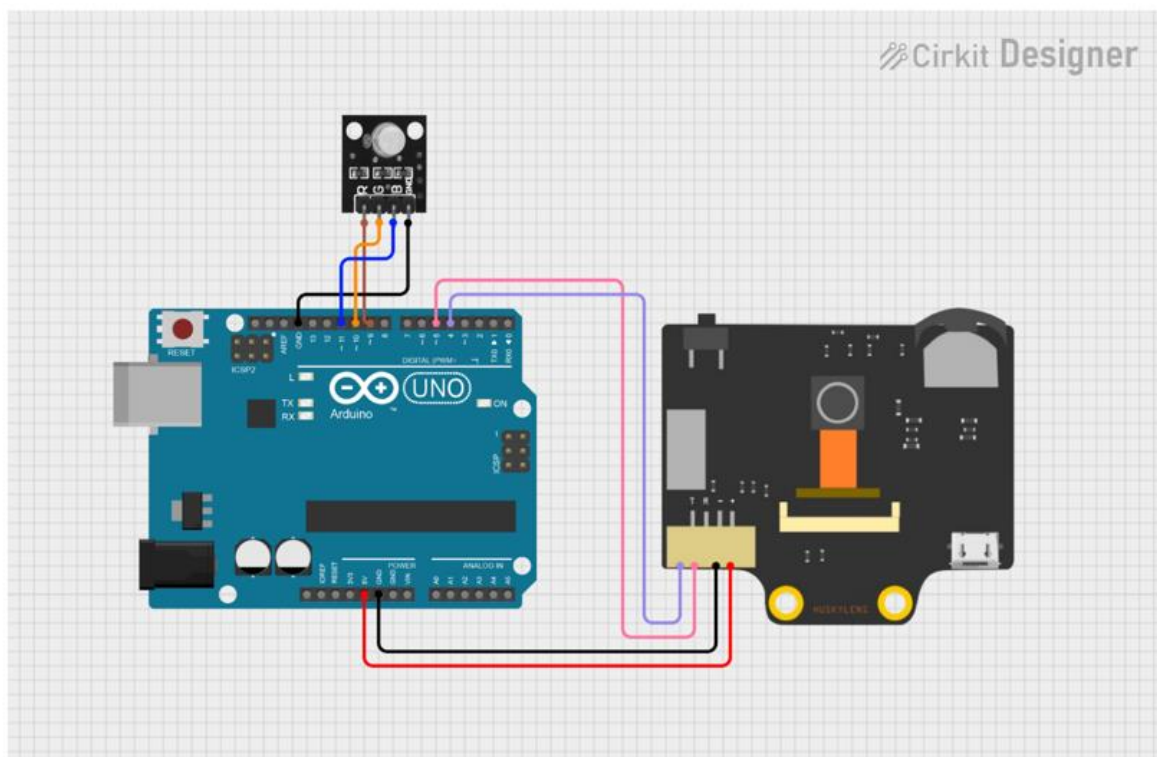
provides immediate text feedback such as "Color RED is detected", it is the moment the AI camera categorizes a learned color within its field of view.

TASK 2: INTEGRATED RGB COLOR DISPLAY

2.0 MATERIALS AND EQUIPMENTS

- Arduino Uno
- USB cable
- Gravity HuskyLens AI Vision Camera
- Jumper wires
- RGB LED - HW479

2.1 EXPERIMENTAL SETUP



Schematic setup using Cirkuit Software

Figure 2.0 shows the experimental setup for task 2

2.2 METHODOLOGY

a. Circuit Assembly

- The VCC is connected to Arduino 5V
- GND to Arduino GND
- TX is connected to Arduino D4
- RX is connected to Arduino D5
- R (Red) is connected to Arduino D9
- G (green) is connected to Arduino D10
- B (Blue) is connected to Arduino D11

b. Programming and uploading

- The Arduino ide is being downloaded and connect it to COM 9

c. Programming Logic

- If the camera detects an object, it reads the specific result.ID; if the detected object is categorized as "Object 1" (Case 1), the LED turns Red.
- If the detected object is categorized as "Object 2" (Case 2), the LED turns Green.
- If the detected object is categorized as "Object 3" (Case 3), the LED turns Blue.
- If the detected object is categorized as "Object 4" (Case 4), the LED turns Yellow.
- If the detected object is categorized as "Object 5" (Case 5), the LED turns Purple.

d. Coding

```

1  #include "HUSKYLENS.h"
2  #include "SoftwareSerial.h"
3
4  HUSKYLENS huskylens;
5  SoftwareSerial mySerial(4, 5); // RX, TX
6
7  // RGB LED Pins (PWM capable)
8  const int redPin = 9;
9  const int greenPin = 10;
10 const int bluePin = 11;
11
12 void setup() {
13   Serial.begin(115200);
14   mySerial.begin(9600);
15
16   // Set RGB pins as outputs
17   pinMode(redPin, OUTPUT);
18   pinMode(greenPin, OUTPUT);
19   pinMode(bluePin, OUTPUT);
20
21   while (!huskylens.begin(mySerial)) {
22     Serial.println(F("Begin failed! Check Protocol (Serial 9600)"));
23     delay(500);
24   }
25 }
26
27 void loop() {
28   if (!huskylens.request()) {
29     setColor(0, 0, 0);
30     return;
31   }
32
33   if (huskylens.available()) {
34     HUSKYLENSResult result = huskylens.read();
35
36     // The Serial Monitor will now show exactly which color is triggered
37     Serial.print("Detected ID: ");
38     Serial.print(result.ID);
39     Serial.print(" - Color: ");
40
41     switch (result.ID) {
42       case 1:
43         Serial.println("RED");
44         setColor(255, 0, 0);
45         break;
46
47       case 2:
48         Serial.println("GREEN");
49         setColor(0, 255, 0);
50         break;
51
52       case 3:
53         Serial.println("BLUE");
54         setColor(0, 0, 255);
55         break;
56
57       case 4:
58         Serial.println("YELLOW");
59         setColor(255, 255, 0);
60         break;
61
62       case 5:
63         Serial.println("PURPLE");
64         setColor(150, 0, 255);
65         break;
66
67       default:
68         Serial.println("UNKNOWN/WHITE");
69         setColor(255, 255, 255);
70         break;
71     }
72   } else {
73     // Optional: Print "Searching..." if you want to see activity when empty
74     setColor(0, 0, 0);
75   }
76 }
77
78 // Helper function to handle PWM values
79 void setColor(int redValue, int greenValue, int blueValue) {
80   analogWrite(redPin, redValue);
81   analogWrite(greenPin, greenValue);
82   analogWrite(bluePin, blueValue);
83 }

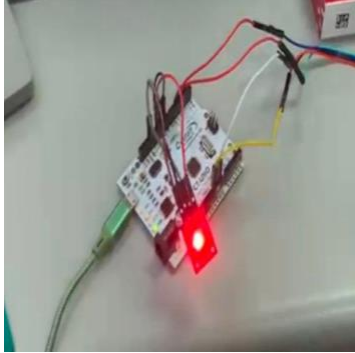
```

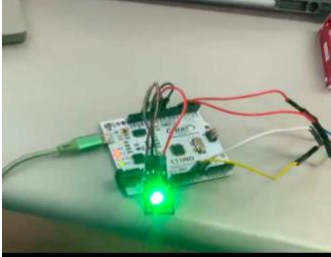
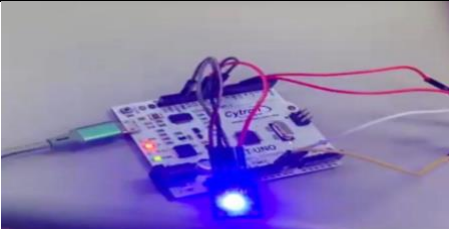
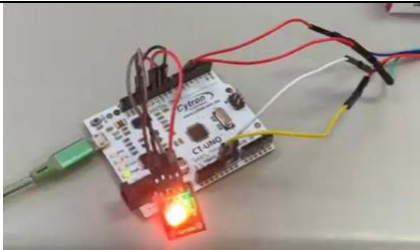
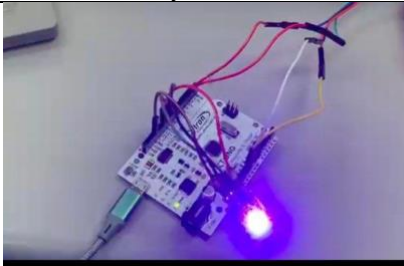
2.3 DATA ANALYSIS

Detected ID	Actual Color presented	Final LED output
1	Red	Red
2	Green	Green
3	Blue	Blue
4	Yellow	Yellow
5	Purple	Purple

Based on the table above, the first detected ID which is “1” is represented as the color Red and the final color of led is Red. This happened when the Husky Lens detect an object with red on it. Next, the detected ID of “2” is representing the color of green and the LED light will light up in green. This is because of the Husky lens detects an object that has mainly green on it. The third detected ID is “3”. When the husky lens detects an object with blue on it, the “3” represents the color of blue and the LED output is blue. On the other hand, when the Husky Lens detects an object with yellow on it, it will detected ID of “4” and the LED will light up on Yellow. Finally, the “5” is represents as the color of Purple and the LED output will show the purple color.

2.4 RESULT

DETECTED ID	COLOR REPRESENTED	LED LIGHT OUTPUT
1	 <u>Figure 2.1 shows the Husky lens detects an object with red color on it</u>	 <u>Figure 2.2 shows the LED lights output is Red</u>

2		
	<u>Figure 2.4 shows the Husky lens detects an object with green color on it</u>	<u>Figure 2.5 shows the LED lights output is Green</u>
3		
	<u>Figure 2.3 shows the Husky lens detects an object with blue color on it</u>	<u>Figure 2.4 shows the LED lights output is Red</u>
4		
	<u>Figure 2.6 shows the Husky lens detects an object with yellow color on it</u>	<u>Figure 2.7 shows the LED lights output is yellow</u>
5		
	<u>Figure 2.8 shows the Husky lens detects an object with purple color on it</u>	<u>Figure 2.9 shows the LED lights output is purple</u>

2.5 DISCUSSION

The integration of the HuskyLens AI vision sensor with an RGB LED display creates a closed-loop feedback system where visual recognition translates directly into physical signals. The HuskyLens acts as the primary input device, handling the "heavy lifting" of machine learning and object classification internally. This allows the Arduino microcontroller to focus on simple logic gates rather than complex image processing. The program is designed for responsiveness; by checking the availability of the HuskeyLens before executing color changes, the system ensures that the LED only reacts when a valid object is firmly within the camera's field of view.

The detected data is converted into specific light frequencies. Using a switch (result.ID) structure is highly efficient for this task, as it provides a clear "if-else" style roadmap for the controller to follow. When the camera identifies "Object 1," the software activates the red pin; when the object is no longer detected, the else condition triggers the setColor(0, 0, 0) command, effectively turning the LED off. This setup is a fundamental example of how AI vision can be used to create interactive environments, providing the user with immediate, color-coded confirmation of the AI's classification accuracy.

3.0 CONCLUSION

In conclusion, Task 1 successfully demonstrated the basic functionality of the HuskyLens in object and color recognition. The system was able to detect a trained object and control an LED using simple if-else logic, while also displaying different color IDs through the Serial Monitor. This shows that the HuskyLens can effectively process visual data and communicate with the Arduino for real-time decision-making, although its performance is influenced by environmental conditions such as lighting and shadows.

For Task 2, the integration of an RGB LED enhanced the system by providing direct visual feedback based on detected color IDs. The results confirmed that each detected color was accurately mapped to its corresponding RGB output, creating a more interactive and responsive system. Overall, this task highlights the practical application of AI vision in embedded systems, where visual inputs can be efficiently translated into meaningful physical outputs.

4.0 RECOMMEENDATION

The primary technical challenges observed during this task involve the sensitivity of the AI vision sensor to environmental variables. Ambient light poses a significant issue by causing color shifting because the HuskyLens identifies colors based on specific saturation and brightness values, external light sources can alter these readings and cause the system to misidentify a learned color. Furthermore, shadow interference creates dark patches on the target object, which directly reduces the sensor's confidence level during detection and can lead to intermittent data reporting in the code.

To mitigate these errors, it is highly recommended to implement a controlled lighting environment by attaching a small white LED to the system, ensuring the target remains illuminated in a neutral, consistent light. Additionally, the system's reliability can be improved through multi-condition training, which involves performing the "learn" function for the same object ID under various lighting and shadowed situations. This approach broadens the sensor's recognition parameters, allowing the programming logic to maintain a stable detection state even when visual conditions are not ideal.

5.0 REFERENCES

HuskyLens. (2023, April 9). *HUSKYLENS Arduino API*. GitHub.

<https://github.com/HuskyLens/HUSKYLENSArduino>

HuskyLens Tutorial - Getting Started with Husky Lens - AI Vision Sensor. (n.d.).

Www.youtube.com. <https://www.youtube.com/watch?v=HZxFj2u9k00>

HuskyLens WIKI Document.pdf. (2019). *HuskyLens WIKI Document.pdf*. Google Docs.

<https://drive.google.com/file/d/1FobcfurW->

[JiH7lUpDhGipDavSHsuWPXZ/view?usp=sharing](https://drive.google.com/file/d/1FobcfurW-JiH7lUpDhGipDavSHsuWPXZ/view?usp=sharing)

Lee, J. (2025, August 21). *Getting Started with HuskeyLens & Arduino*. Cytron Technologies

Malaysia; Cytron Technologies. [https://my.cytron.io/tutorial/getting-started-with-](https://my.cytron.io/tutorial/getting-started-with-huskylens-and-arduino)

[huskylens-and-arduino](https://my.cytron.io/tutorial/getting-started-with-huskylens-and-arduino)

6.0 ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin and the lab technicians, for all their help, encouragement and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

7.0 STUDENTS DECLARATION

We hereby declare that, except for the places where it is acknowledged, all the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behaviour. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein has not been done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributor to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final report has been verified by us.

Signature: *nurulanis*

Name: NURUL ANIS BINTI MOHD KHAIRI

Matric Number: 2419788

Read ☐

Understand ☐

Agree ☐

Signature: *athirahmeor*

Name: WAN NUR ATHIRAH BINTI MEOR MOHD REDZUAN

Matric Number: 2320102

Read ☐

Understand ☐

Agree ☐

Signature: *nurkhalisah*

Name: SITI NURKHALISAH BINTI NAHZAMRI

Matric Number: 2328482

Read ☐

Understand ☐

Agree ☐